2, 10 3, 1 6, 2. 8, 11

(1, 0) (2, 4), (2, 1), (2, 5), (3, 2), (4, 3)

Условие. Дан массив вещественных чисел $A[1:n]$. Для каждого элемента $A[i]$ требуется найти индекс ближайшего элемента справа $j > i$ такого что

$$A[j] \geq 2 \cdot A[i].$$

Если такого элемента не существует вернуть **None**. Ограничения: время $O(n \log n)$, память $O(n)$.

idx	1	2	3	4	5
A	2	3	4	1	2
ans	3	–	–	5	–

i

в лоб: $O(n^2)$

4

(2, 1) ← куча min h

(∃ л-ты, для кот не зн. ответ)

зн idx

ans = [None] · n

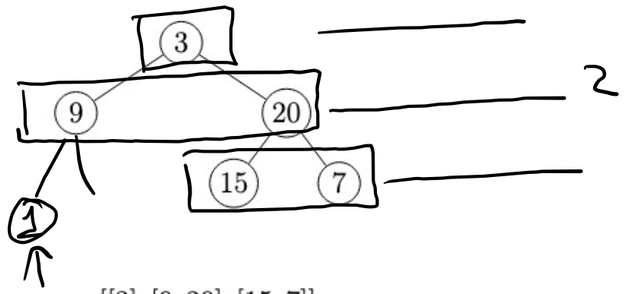
(A[i] не явл. отв. для h[j])

```
while len(h)>0 & A[i] ≥ 2·h[0][0]:
        ans[h.heappop()[1]] = i
h.heappush((A[i], i))
```

Условие. Дан корень бинарного дерева. Требуется вернуть обход дерева по уровням (level order traversal), то есть значения узлов слева направо, уровень за уровнем.

Если дерево пустое, вернуть пустой список.

Пример.



Ответ.

[[3], [9, 20], [15, 7]]

[3 , 9, 20, 15, 7]

[~~9~~ 20]

[9, 20]

[3]

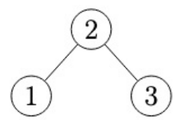
[15 7]

[~~9~~ 20 7 1]

[~~9~~ , ~~20~~ , 1 , 15 , 7]

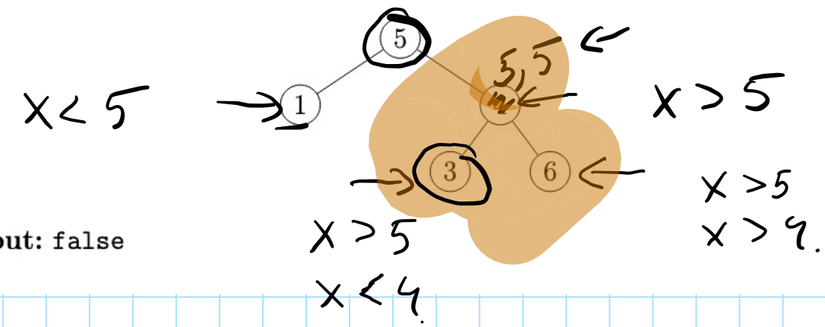
Условие. Дан корень бинарного дерева. Определить, является ли это дерево корректным бинарным деревом поиска (BST).

Пример 1.



Output: `true`

Пример 2.



Output: `false`

V+E

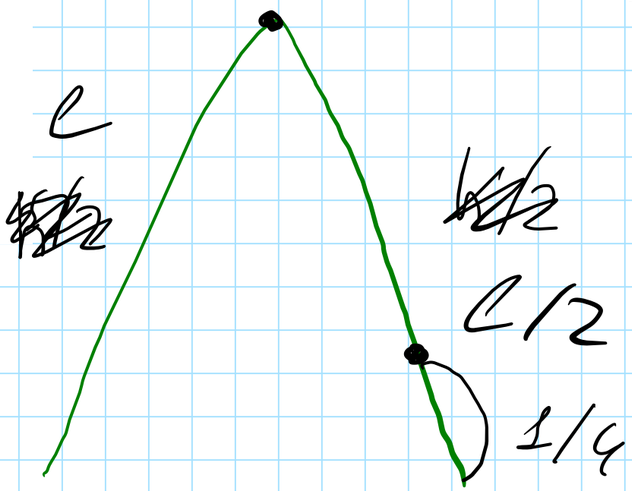
Задан граф в виде дерева с n вершинами, пронумерованными от 0 до $n-1$, и массив из $n-1$ рёбер, где `edges[i]` $= [a_i, b_i]$ обозначает наличие неориентированного ребра между вершинами a_i и b_i.

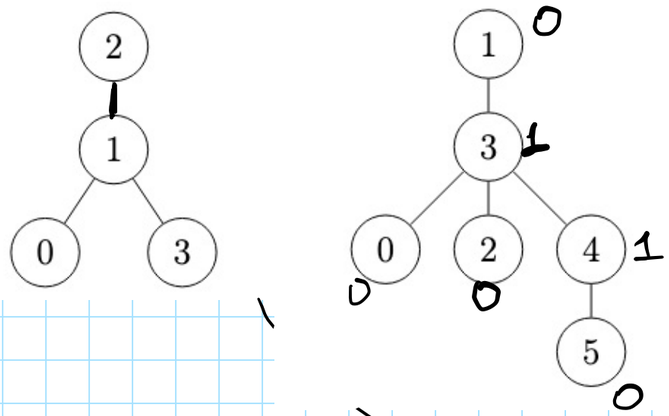
Можно выбрать любую вершину в качестве корня дерева. При выборе вершины x в качестве корня дерево получает некоторую высоту h. Необходимо найти все такие вершины, которые при выборе в качестве корня образуют дерево минимальной высоты h.

Верните список всех таких вершин. Ответ может быть в произвольном порядке.

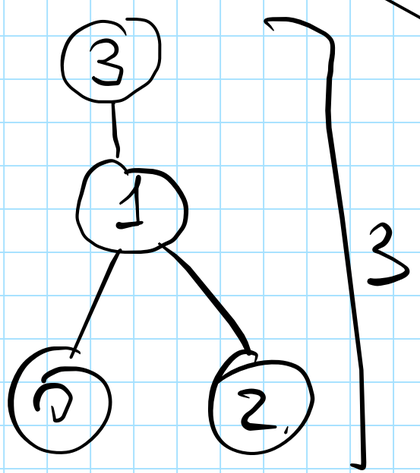
1 : 7

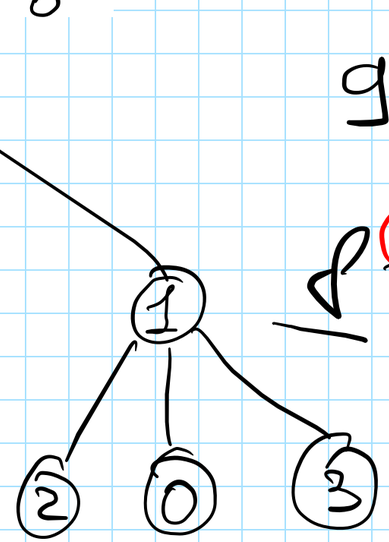
7 : 1, 2, 3, 4, 5, 6

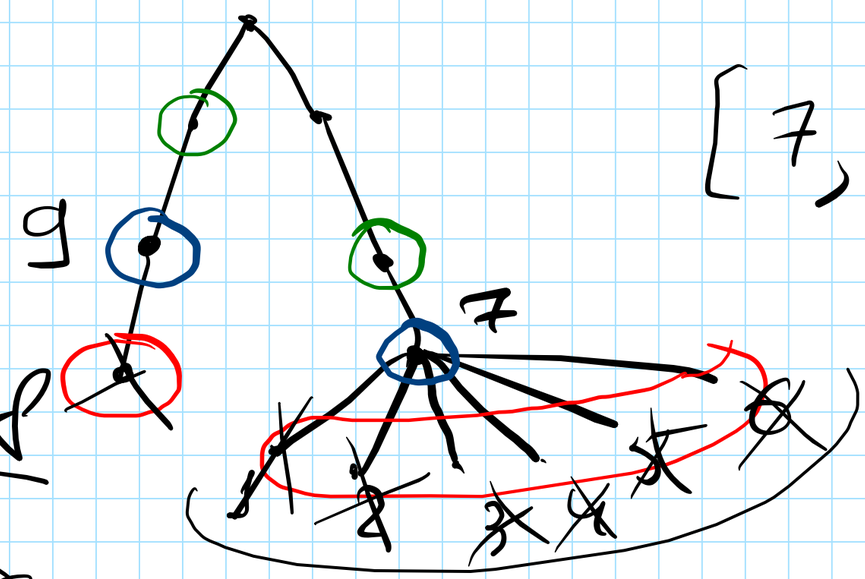




3, 4.







[7, 9]